

Tabla de contenido

Comando find	1
Sintaxis de find	1
Búsquedas básicas	2
Búsquedas a través del tiempo.....	2
Comparaciones con -and, -or y -not	3
El tamaño si importa.....	3
Comando grep	4
Enlace simbólico	5
Enlaces duros.....	6
Eliminar enlaces.....	6
Actividades.....	6

Comando find

El comando `find` de Linux es extremadamente potente, esto es, si logras usarlo adecuadamente. No hay nada mejor para hacer todo tipo de búsquedas de archivos y carpetas que este comando.

Hay por supuesto otros comandos de búsqueda como `awk`, `sed` y `grep` pero están más enfocados a buscar "dentro" de los archivos. `find` es mucho más útil para encontrar archivos y directorios. En este artículo aprenderás a usar `find` como todo un experto y después puedas aplicarlo en usos administrativos de todo tipo.

Sintaxis de find

La sintaxis es muy simple:

```
find [ruta] [expresión_de_búsqueda] [acción]
```

La `[ruta]` es cualquier directorio o path que se quiera indicar y desde donde inicia la búsqueda, ejemplos pueden ser `"/etc"`, `"/home/sergio"`, `"/"`, `"."` si no se indica una ruta se toma en cuenta entonces el directorio donde se este actualmente, es decir el directorio de trabajo actual, que es lo mismo que indicar punto `"."`. De hecho es posible indicar más de un directorio de búsqueda como se verá más adelante en un ejemplo.

La `[expresión_de_búsqueda]` es una o más opciones que puede devolver la búsqueda a realizar en si o acciones a realizar sobre la búsqueda, si no se indica ninguna expresión de búsqueda se aplica por defecto la opción `-print` que muestra el resultado de la búsqueda.

La `[acción]` es cualquier comando de Linux invocado a ejecutarse sobre cada archivo o directorio encontrado con la `[expresión_de_búsqueda]`.

Los tres argumentos anterior son enteramente opcionales

Búsquedas básicas

El siguiente ejemplo busca todos los archivos que contengan en su nombre "reporte" desde la raíz:

```
find / -name reporte

find / -iname Reporte (lo mismo, pero sin tomar en cuenta
mayúsculas y minúsculas)
```

El uso de expresiones regulares en lo que se busca es válido:

```
find / -name "[0-9]*" (todo lo que empiece con un dígito)
find / -name "[Mm]*" (todo lo que empiece con un la letra M o
m)
find / -name "[a-m]*.txt" (todo lo que empiece entre a y m y
termine en ".txt")
```

Busca bajo /home todos los archivos que pertenezcan al usuario mario

```
find /home -user mario

(lo mismo y que contengan con "enero" como en reporte_enero2011)
find /home -user mario -name "*enero*"
```

No estás limitado a un solo directorio, indica más de uno a buscar antes de las expresiones:

```
find /etc /usr /var -group admin

(busca en tres directorios todos los archivos o
subdirectorios que pertenezcan al grupo 'admin')
```

Búsquedas a través del tiempo

Varias opciones aceptan argumentos numéricos, estos pueden ser indicados de tres maneras posibles:

```
+n busca valores mayor que n
-n busca valores menor que n
n busca exactamente el valor n
```

Buscar todos los archivos que hayan cambiado en los últimos 30 minutos:

```
find / -mmin -30 -type f

los modificados exactamente hace 30 minutos:
find / -mmin 30 -type f
```

O si deseas buscar en un rango específico de minutos, con este ejemplo buscarías todos los directorios que hayan cambiado hace más de 10 minutos (+10) y menos de 30 (-30)

```
find / -mmin +10 -mmin -30 -type d

aunque lo anterior sería mas exacto decir los modificados hace 11 minutos o más
y 29 minutos o menos, ya que como se vio anteriormente +n y -n indican
"mayor que" y "menor que", el ejemplo correcto sería entonces:
find / -mmin +9 -mmin -31 -type d
```

find ofrece varias opciones de búsqueda por tiempo, pero las principales son: -amin, -atime, -cmin, ctime, -mmin y -mtime. "min" es para periodos de minutos y "time" para periodos de 24 horas.

Los que empiezan con "a" (access) indica el tiempo en que fue accedido (leído) por última vez un archivo. Los que empiezan con "c" (change) indica el tiempo que cambió por última vez el status de un archivo, por ejemplo sus permisos. Los que empiezan con "m" (modify) indica el tiempo en que fue modificado (escrito) por última vez un archivo.

Una consideración a tener con las búsquedas -atime, -ctime y -mtime es que el tiempo se mide en periodos de 24 horas y estos son siempre truncados, con ejemplos es más claro:

```
find . -mtime 0 (busca archivos modificados entre ahora y hace un dia)
find . -mtime -1 (busca archivos modificados hace menos de un dia)
find . -atime 1 (busca archivos accedidos entre hace 24 y 48 horas)
find . -ctime +1 (busca archivos cuyo status haya cambiado hace más de 48 horas)
```

Comparaciones con -and, -or y -not

find también incluye operadores booleanos que la hace una herramienta aun más útil:

```
find /home -name 'ventas*' -and -mmin 120
find /home -name 'reporte[_-]*' -not -user sergio
find /home -iname '*enero*' -or -group gerentes
```

El primer ejemplo busca todos los archivos que comiencen con 'ventas' Y que hayan sido modificados o cambiados en las últimas dos horas (120 minutos).

El segundo ejemplo busca todos los archivos que comiencen con 'reporte' y después siga un _ o un - y que NO pertenezcan al usuario sergio.

El tercer ejemplo busca todos los archivos que contengan la palabra enero, Enero, ENERO, etc. (sin importar si lleva mayúsculas o minúsculas) O cualquier otro archivo que encuentre que pertenezca al grupo 'gerentes'.

Estas opciones de booleanos tienen su correspondiente abreviatura:

- and se puede indicar también como -a
- or se puede indicar también como -o
- not se puede indicar también como !

El tamaño si importa

Una de las actividades básicas de un administrador de sistemas Linux es monitorear el tamaño de archivos, sobre todo de usuarios. Con `find` es muy fácil realizar búsquedas por tamaño, se indica con la opción `-size`, se aplican las mismas reglas para argumentos numéricos (`+n -n n`).

```
find /var/log -size +15000k -name "*.jpg" (busca archivos mayores a 15 megas del tipo jpg)
find $HOME -800c (busca en tu home todos los archivos menores a 800 bytes (799 realmente))

(archivos de tamaño comprendidos entre 1mb y 10mb)
find . -size +1000k -and -size -10000k
```

Se admiten cuatro parámetros después del número en `-size`:

c = bytes
w = 2 byte words
k = kilobytes
b = 512-byte bloques

Para buscar archivos vacíos puedes entonces hacer lo siguiente:

```
find . -size 0c

(Aunque la opción -empty hace lo mismo más eficientemente)
find . -empty
```

Comando grep

El comando **grep** nos permite buscar, dentro de los archivos, las líneas que concuerdan con un patrón. Bueno, si no especificamos ningún nombre de archivo, tomará la entrada estándar, con lo que podemos encadenarlo con otros filtros.

Por defecto, `grep` imprime las líneas encontradas en la salida estándar. Es decir, que podemos verlo directamente la pantalla, o redireccionar la salida estándar a un archivo.

Como tiene muchísimas opciones, vamos a ver tan sólo las más usadas:

- c En lugar de imprimir las líneas que coinciden, muestra el número de líneas que coinciden.
- e PATRON nos permite especificar varios patrones de búsqueda o proteger aquellos patrones de búsqueda que comienzan con el signo `-`.
- r busca recursivamente dentro de todos los subdirectorios del directorio actual.
- v nos muestra las líneas que no coinciden con el patrón buscado.
- i ignora la distinción entre mayúsculas y minúsculas.
- n Numera las líneas en la salida.
- E nos permite usar expresiones regulares. Equivalente a usar **egrep**.
- o le indica a `grep` que nos muestre sólo la parte de la línea que coincide con el patrón.
- f ARCHIVO extrae los patrones del archivo que especifiquemos. Los patrones del archivo deben ir uno por línea.

-H nos imprime el nombre del archivo con cada coincidencia.

Veamos algunos ejemplos:

- Buscar todas las líneas que contengan palabras que comiencen por **a** en un archivo:

```
$ grep '\<a.*\>' archivo
```

Otra forma de buscar, sería:

```
$ cat archivo | grep "\<a.*\>"
```

- Mostrar por pantalla, las líneas que contienen comentarios en el archivo /boot/grub/menu.lst:

```
$ grep "#" /boot/grub/menu.lst
```

- Enviar a un fichero las líneas del archivo /boot/grub/menu.lst que no son comentarios:

```
$ grep -v "#" /boot/grub/menu.lst
```

- Contar el número de interfaces de red que tenemos definidos en el fichero /etc/network/interfaces:

```
$ grep -c "iface" /etc/network/interfaces
```

- Mostrar las líneas de un fichero que contienen la palabra BADAJOZ o HUELVA:

```
$ grep -e "BADAJOZ" -e "HUELVA" archivo
```

- Mostrar las líneas de un fichero que contienen la palabra BADAJOZ o HUELVA, numerando las líneas de salida:

```
$ grep -n -e "BADAJOZ" -e "HUELVA" archivo
```

- Mostrar los ficheros que contienen la palabra TOLEDO en el directorio actual y todos sus subdirectorios:

```
$ grep -r "TOLEDO" *
```

Veamos algunos ejemplos con expresiones regulares:

- Obtener la dirección MAC de la interfaz de red eth0 de nuestra máquina:

```
$ ifconfig eth0 | grep -oiE '([0-9A-F]{2}:){5}[0-9A-F]{2}'
```

Sacamos la dirección MAC de la interfaz eth0 de nuestra máquina haciendo un: **ifconfig eth0**

Y aplicando el filtro grep:

```
grep -oiE '([0-9A-F]{2}:){5}[0-9A-F]{2}'
```

Un enlace simbólico es un acceso a un directorio o fichero que se encuentra en un lugar distinto dentro de la estructura de directorios (una especie de acceso directo). Una modificación realizada utilizando este enlace se reflejará en el original; pero, por el contrario, si se elimina el enlace, no se eliminará el auténtico.

Cualquier cambio que se haga tanto en el directorio original como en el contenido del enlace simbólico quedará reflejado en el otro (si cambias el contenido del enlace simbólico se podrá ver en el original y viceversa)

Borrar el enlace simbólico no afecta al original (seguirá existiendo)

Para crear un enlace simbólico lanzaremos el siguiente comando desde la terminal:

```
ln -s DIRECTORIO_ORIGINAL DIRECTORIO_ENLACE
```

NOTA: Para recordar el parámetro -s asociar la palabra SOFT o SIMBÓLICO seguro que así no se te olvida.

Enlaces duros

Igual que el simbólico pero con un matiz: cuando se borre el último enlace que apunte al DIRECTORIO_ORIGINAL (se pueden crear tantos enlaces como se necesiten a un directorio cualquiera de nuestro sistema de archivos) se borrará el DIRECTORIO_ORIGINAL

La sintaxis es la misma cambiando única y exclusivamente el parámetro por -t; sirva de ejemplo

Eliminar enlaces

Tan sencillo como ejecutar el comando unlink

```
unlink RUTA_AL_ENLACE_NO_DESEADO
```

Actividades

1. Redacta los comandos necesarios para obtener (find):
 - a. Busca en el sistema todos los ficheros txt de tu equipo:
 - b. Busca en el sistema todos los archivos del usuario root
 - c. Busca en el sistema todos los directorios que empiezan por la letra e.
 - d. Busca los archivos del sistema que se han modificado en el periodo comprendido entre hace una hora y media hora.
 - e. Busca los ficheros menores de un megabyte en la carpeta personal de uno de tus usuarios.
 - f. Busca todos los ficheros del sistema que no pertenezcan a root y que ocupen menos de 2 megabytes.
2. Redacta los comandos necesarios para obtener (grep):
 - a. Muestra la información asociada al grupo root que contiene el fichero /etc/group
 - b. Muestra la información asociada a un usuario (el que utilizas habitualmente) que contiene el fichero /etc/passwd